AGENTS.md — AI Assistant Conventions for Colab Agent

Project Identity

This is **Colab Agent**: an LLM-powered autonomous agent that fine-tunes HuggingFace models in Google Colab. It plans, generates code, executes in Colab, parses results, and iterates — with runtime auto-switching, error recovery, cost tracking, a plugin system, and self-improvement.

Critical Rules

Triple-quote nesting

Inside `return f"""..."""`, ALWAYS use `'''` for inner triple-quoted strings (docstrings, SQL, generated code). Wrong: `f"""...\"\"\"..."""` — this prematurely closes the outer string.

F-string escaping

Inside an outer `f"""..."""`, `{...}` is an expression placeholder. To produce literal `{...}` in the output (e.g. for variables in generated Python code), use `{{...}}`.

Import shadowing

The directory `gh_integration/` replaces an old `github/` directory. Do NOT create or reference `github/`. Use `from gh_integration.integration import GitHubIntegration`.

Mocking GitHub in tests

Use `@patch("github.Github")` (the library module), NOT `@patch("gh_integration.integration.Github")`. PyGithub's `Github` class is imported inside `_get_client()`.

FastAPI over stdlib

`python -m agent.service --fastapi` for production (uvicorn). `python -m agent.service` for zero-dep fallback (stdlib HTTPServer). `python cli.py serve` defaults to FastAPI.

File Conventions

Path	Convention
`agent/core.py`	Main `OrchestratorAgent` class. Sync `run()` + async `async_run()`. Both must be kept in sync.
`agent/plugin.py`	`Plugin` base class + `PluginRegistry`

Path	Convention
	(thread-safe) + `HookRunner` (error-isolated).
`agent/llm_cache.py`	SQLite-backed cache. Thread-safe. WAL mode. `LLMCachePlugin` for auto-registration.
`agent/self_improvement.py`	Error classifier + `SelfImprovementPlugin` (auto-registered, priority 50).
`agent/safety.py`	`CostTracker`, `BudgetManager`, code sanitizers. Thread-safe.
`config/settings.py`	Pydantic v2 settings. `env` field controls dev/staging/prod. Prod requires API key + PG.
`tests/test_plugin.py`	Tests for plugin system + LLM cache.
`tests/test_self_improvement.py`	Tests for error classification + self-improvement + async core.

Testing

- 554 tests pass (1 pre-existing timeout in test_models.py due to transformers stdlib metadata issue)
- Quick test run: `pytest tests/test_plugin.py tests/test_self_improvement.py tests/test_config.py tests/test_api.py -v`
- All tests: `pytest tests/ --ignore=tests/test_models.py -q`
- Ruff: `ruff check . --exclude capabilities --exclude .github`
- GitHub mock pattern: `@patch("github.Github")` not `@patch("gh_integration.integration.Github")`
- LLM mock pattern: use `mock_llm_client` fixture (defined in tests/conftest.py). Patches LLMClient on agent.llm_client and agent.core.
- Integration/e2e tests now pass without live APIs: 7 tests in test_integration.py + 3 in test_e2e_agent.py all use mocks.
- Conftest provides env defaults (GH/HF tokens, DB URL, API keys) for all tests.
- Storage tests use `_fresh_db()` (reset_session + init_db) per class for isolation.

Production Hardening Requirements

Every module must have:

- `threading.Lock` or `RLock` on all shared mutable state
- Input validation with `TypeError` on type mismatch

- Error isolation (one component failure doesn't cascade)
- Graceful degradation (safe defaults on non-critical errors)
- Resource cleanup (`close()` methods, context managers)
- Thread safety tests (20+ concurrent workers)
- Malformed/edge-case input tests

Key Decisions

- **NVIDIA API via OpenAI provider**: Use `COLAB_AGENT_OPENAI_BASE_URL=https://integrate.api.nvidia.com/v1` + `COLAB_AGENT_LLM_PROVIDER=openai`
- **Fast LLM model**: `meta/llama-3.1-8b-instruct` (1–5s/call) for development; swap to stronger models for production
- **Test key (Fernet)**: `z8lFvwvawH-vjarAXB6H5KG-iYNQA5lDnzZrsJi_mMs=`
- **Config singleton**: `config.settings.settings` is lazily loaded; call `load_settings()` to force init
- **Plugin logging**: Always catch `Exception` in hook methods; log with `logger.error(f"Plugin {p.name} {hook} failed: {e}")`